



Pair with Target Sum (easy)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
 - Code
 - Time Complexity
 - Space Complexity
- An Alternate approach
 - Time Complexity
 - Space Complexity

Problem Statement

Given an array of sorted numbers and a target sum, find a **pair in the array whose sum is equal to the given target**.

Write a function to return the indices of the two numbers (i.e. the pair) such that they add up to the given target.

Example 1:

Input: [1, 2, 3, 4, 6], target=6
Output: [1, 3]
Explanation: The numbers at index 1 and 3 add up to 6: 2+4=6

Example 2:

Input: [2, 5, 9, 11], target=11
Output: [0, 2]
Explanation: The numbers at index 0 and 2 add up to 11: 2+9=11

Try it yourself

Try solving this question here:



```
1 class PairWithTargetSum {
2
3     public static int[] search(int[] arr, int targetSum) {
4         int p1 = 0 , p2 = arr.length-1;
5         while(p1<p2){
```



```

6      int sum = arr[p1] + arr[p2];
7      if( sum == targetSum)
8          return new int[]{p1, p2};
9      if(sum > targetSum)
10         p2--;
11     else
12         p1++;
13
14     }
15     return new int[] { -1, -1 };
16 }
17 }

```

Test

Save

Reset



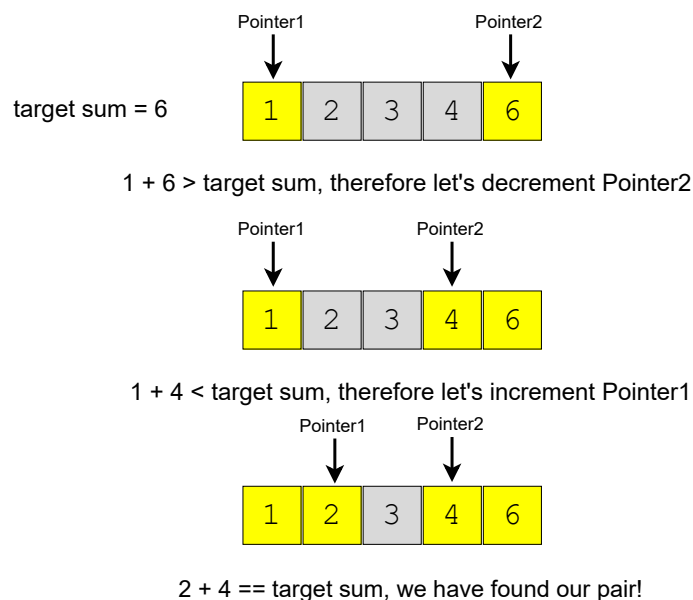
Solution

Since the given array is sorted, a brute-force solution could be to iterate through the array, taking one number at a time and searching for the second number through **Binary Search**. The time complexity of this algorithm will be $O(N * \log N)$. Can we do better than this?

We can follow the **Two Pointers** approach. We will start with one pointer pointing to the beginning of the array and another pointing at the end. At every step, we will see if the numbers pointed by the two pointers add up to the target sum. If they do, we have found our pair; otherwise, we will do one of two things:

1. If the sum of the two numbers pointed by the two pointers is greater than the target sum, this means that we need a pair with a smaller sum. So, to try more pairs, we can decrement the end-pointer.
2. If the sum of the two numbers pointed by the two pointers is smaller than the target sum, this means that we need a pair with a larger sum. So, to try more pairs, we can increment the start-pointer.

Here is the visual representation of this algorithm for Example-1:



Code

Here is what our algorithm will look like:

>

Java

Python3

C++

JS

```

1  class PairWithTargetSum {
2
3      public static int[] search(int[] arr, int targetSum) {
4          int left = 0, right = arr.length - 1;
5          while (left < right) {
6              int currentSum = arr[left] + arr[right];
7              if (currentSum == targetSum)
8                  return new int[] { left, right }; // found the pair
9
10             if (targetSum > currentSum)
11                 left++; // we need a pair with a bigger sum
12             else
13                 right--; // we need a pair with a smaller sum
14         }
15         return new int[] { -1, -1 };
16     }
17
18     public static void main(String[] args) {
19         int[] result = PairWithTargetSum.search(new int[] { 1, 2, 3, 4, 6 }, 6);
20         System.out.println("Pair with target sum: [" + result[0] + ", " + result[1] + "]");
21         result = PairWithTargetSum.search(new int[] { 2, 5, 9, 11 }, 11);
22         System.out.println("Pair with target sum: [" + result[0] + ", " + result[1] + "]");
23     }
24 }

```

Run

Save

Reset

Time Complexity#

The time complexity of the above algorithm will be $O(N)$, where 'N' is the total number of elements in the given array.

Space Complexity

The algorithm runs in constant space $O(1)$.

An Alternate approach

Instead of using a two-pointer or a binary search approach, we can utilize a **HashTable** to search for the required pair. We can iterate through the array one number at a time. Let's say during our iteration we are at number 'X', so we need to find 'Y' such that " $X + Y == Target$ ". We will do two things here:

1. Search for 'Y' (which is equivalent to " $Target - X$ ") in the **HashTable**. If it is there, we have found the required pair.
2. Otherwise, insert "X" in the **HashTable**, so that we can search it for the later numbers.



Here is what our algorithm will look like:

Java Python3 C++ JS

```
import java.util.HashMap;

class PairWithTargetSum {

    public static int[] search(int[] arr, int targetSum) {
        HashMap<Integer, Integer> nums = new HashMap<>(); // to store numbers and their indices
        for (int i = 0; i < arr.length; i++) {
            if (nums.containsKey(targetSum - arr[i]))
                return new int[] { nums.get(targetSum - arr[i]), i };
            else
                nums.put(arr[i], i); // put the number and its index in the map
        }
        return new int[] { -1, -1 }; // pair not found
    }

    public static void main(String[] args) {
        int[] result = PairWithTargetSum.search(new int[] { 1, 2, 3, 4, 6 }, 6);
        System.out.println("Pair with target sum: [" + result[0] + ", " + result[1] + "]");
        result = PairWithTargetSum.search(new int[] { 2, 5, 9, 11 }, 11);
        System.out.println("Pair with target sum: [" + result[0] + ", " + result[1] + "]");
    }
}
```

Run Save Reset

Time Complexity

The time complexity of the above algorithm will be $O(N)$, where 'N' is the total number of elements in the given array.

Space Complexity

The space complexity will also be $O(N)$, as, in the worst case, we will be pushing 'N' numbers in the **HashTable**.

[← Back](#)

[Next →](#)

Introduction

Remove Duplicates (easy)